UNIVERSIDADE TECNOLÓGICA FEDERAL DO PARANÁ

VICTOR ANGELO SOUZA SANTOS

UM ESTUDO SOBRE A IMPLEMENTAÇÃO DE UM FIREWALL INTELIGENTE PARA BLOQUEIO DE CASAS DE APOSTAS ONLINE EM AMBIENTES DE BAIXOS RECURSOS

CAMPO MOURÃO
2026

RESUMO

O cenário contemporâneo da segurança cibernética exige soluções proativas para mitigar o acesso a plataformas de apostas online, cujos domínios dinâmicos e tráfego criptografado via HTTPS(Hypertext Transfer Protocol) tornam os métodos de filtragem por listas estáticas obsoletas. Embora o Aprendizado de Máquina (*Machine Learning*) ofereça alta precisão na classificação de tráfego comportamental, sua implementação em *firewalls* geralmente demanda infraestruturas de hardware robustas, o que impõe barreiras para a adoção em redes de borda ou dispositivos de baixo desempenho. Este trabalho propõe investigar a viabilidade técnica de um firewall inteligente baseado em um comitê de votação majoritária (*Ensemble Learning*), composto pelos algoritmos *XGBoost,* Random Forest e Regressão Logística, operando sob restrições computacionais. A metodologia fundamenta-se na criação de um laboratório virtualizado via containers Docker, onde limites de CPU e memória são impostos para simular hardware de baixo custo. O processamento dos dados utilizara metadados estatísticos de fluxos bidirecionais e a técnica de Incerteza Simétrica (*Symmetrical Uncertainty*) para redução de dimensionalidade, visando otimizar a eficiência da inferência. Os protocolos de teste contemplam métricas de acurácia, Logloss e AUC, integradas a *benchmarks* de engenharia como latência de decisão e *footprint* de memória. Espera-se que os resultados quantifiquem o equilíbrio entre a precisão preditiva e o custo computacional, validando uma arquitetura híbrida assistida por camadas de moderação manual para mitigar o erro residual e assegurar a continuidade dos serviços em redes operacionais reais.

Palavras-chave: firewall inteligente; *machine learning*; segurança de redes; baixos recursos; ensemble learning.

ABSTRACT

The contemporary cybersecurity landscape demands proactive solutions to mitigate access to online gambling platforms, whose dynamic domains and HTTPS encryption render static filtering methods obsolete. Although Machine Learning (ML) offers high precision in behavioral traffic classification, its implementation in firewalls typically requires robust hardware infrastructures, which imposes barriers to adoption in edge networks or low-performance devices. This study investigates the technical feasibility of a smart firewall based on a majority voting ensemble, composed of XGBoost, Random Forest, and Logistic Regression algorithms, operating under computational constraints. The methodology is based on the creation of a virtualized laboratory via Docker containers, where CPU and RAM limits are imposed to simulate low-availability hardware. Data processing utilizes bi-directional flow metadata and the Symmetrical Uncertainty

(SU) technique for dimensionality reduction, aiming to optimize inference efficiency. Testing protocols include accuracy, Logloss, and AUC metrics, integrated with engineering benchmarks such as decision latency and memory footprint. The results are expected to quantify the trade-off between predictive precision and computational cost, validating a hybrid architecture assisted by manual moderation layers to mitigate residual error and ensure service continuity in real operational networks.

Keywords: smart firewall; machine learning; network security; low-resource; ensemble learning.

Sumario

RESUMO....2
ABSTRACT....2
Sumario....3
1 INTRODUÇÃO....4
1.1 PROBLEMÁTICA....4
1.2 OBJETIVO....5
1.3 JUSTIFICATIVA....5
2 REFERENCIAL TEÓRICO....5
2.1 Evolução dos Sistemas de Detecção e Prevenção de Intrusão (IDS/IPS)....5
2.2 Classificação de Tráfego de Rede (NTC) e Criptografia....5
2.3 Aprendizado de Máquina e Seleção de Atributos em Baixos Recursos....6
2.4 Fundamentação do Ensemble Learning: Bagging e Boosting....6
2.5 Métricas de Performance e o Problema do Mundo Aberto....7
3 Metodologia....7
3.1. Arquitetura e Simulação de Ambiente....7
3.1.1. Topologia de Rede Virtualizada via Docker....7
3.1.2. Camadas de Defesa e Subsistemas....8
3.1.3. Metodologia de Restrição de Recursos....9
3.1.4. Instrumentação e Coleta de Métricas....9
3.2. Fluxo de Dados e Seleção de Atributos....10
3.2.1. Aquisição e Rotulagem (Data Acquisition)....10
3.2.2. Engenharia de Atributos Baseada em Fluxos....10
3.2.3. Seleção de Atributos via Incerteza Simétrica (SU)....10
3.2.4. Pré-processamento e Balanceamento....11
3.3. Arquitetura do Comitê de Votação (Ensemble Learning)....12
3.3.1. Especificações dos Classificadores Integrantes....12
3.3.2. Mecanismo de Decisão e Lógica de Consenso....13

3.3.3. Restrições e Ajustes para Baixos Recursos..... 13
3.3.4. Ceticismo sobre a Generalização e o Mundo Aberto..... 13
3.4. Protocolo de Testes e Benchmarking de Performance..... 14
3.4.1. Avaliação de Eficiência Computacional (Footprint)..... 14
3.4.2. Latência de Inferência e Condição de Corrida (Race Condition)..... 14
3.4.3. Teste de Estresse e Ponto de Saturação..... 14
3.4.4. Validação Estatística sob Restrição (Trade-off)..... 15
3.4.5. Critérios de Rejeição da Hipótese..... 15
3.5. Governança e Mitigação do "Gap Semântico"..... 15
3.5.1. Camada de Pré-filtragem e Whitelists..... 16
3.5.2. Interface de Moderação e Interpretabilidade..... 16
3.5.3. Ciclo de Feedback e Correção de Erros (Feedback Loop)..... 16
3.5.4. Limitações da Supervisão Humana..... 16
4. RESULTADOS ESPERADOS..... 17
4.1 Desempenho e Robustez de Classificação da IA..... 17
4.2 Viabilidade em Ambientes de Baixos Recursos..... 17
4.3 Impacto na Latência de Rede (T-Verdict)..... 18
4.4 Eficiência da Governança Automatizada..... 18
Tabela: Cronograma de Execução de Atividades (TCC 2)..... 18
4.5 Plano de Discussão e Análise dos Resultados..... 20
4.5.1 Análise do Compromisso entre Dimensionalidade e Acurácia (Trade-off)..... 20
4.5.2 Avaliação da Eficiência do Comitê frente a Modelos Isolados..... 20
4.5.3 Validação do Impacto Operacional e Restrição de Recursos..... 20
5. CONSIDERAÇÕES FINAIS..... 21
REFERÊNCIAS..... 22

1 INTRODUÇÃO

O cenário contemporâneo da segurança de redes enfrenta um desafio persistente com a sofisticação das ameaças e a mudança nos padrões de tráfego, onde a eficácia da inspeção de pacotes tradicional é frequentemente limitada. Embora o uso de Aprendizado de Máquina (ML) em firewalls seja uma realidade industrial, essas soluções geralmente demandam infraestruturas de hardware robustas para processar fluxos criptografados via HTTPS(Hypertext Transfer Protocol) em tempo real sem degradar a experiência do usuário.

1.1 PROBLEMÁTICA

A rápida ascensão e a popularização das plataformas de apostas online, combinadas com a evolução recente das ferramentas de Inteligência Artificial, criaram um cenário dinâmico que desafia as infraestruturas tradicionais de segurança de redes. Diante desse panorama recente, métodos convencionais de filtragem estática tornaram-se obsoletos, exigindo abordagens inteligentes e baseadas em comportamento.

O núcleo da problemática reside no fato de que, por se tratar de um fenômeno tecnológico extremamente recente, ainda há uma lacuna de estudos focados na otimização e na verificação da aplicabilidade prática desses modelos de Aprendizado de Máquina em cenários de recursos computacionais mínimos. Torna-se necessário, portanto, investigar como viabilizar a execução dessas arquiteturas preditivas complexas em hardwares limitados, permitindo a criação de sistemas de *firewall* que ofereçam bloqueios mais eficientes a esse ecossistema de apostas e que, ao mesmo tempo, sejam economicamente acessíveis e replicáveis para pequenas instituições.

1.2 OBJETIVO

Este projeto propõe-se a desenvolver e avaliar a viabilidade técnica de um *firewall* inteligente no contexto de bloqueio de casas de apostas online em ambientes de baixo recurso. Pretende-se verificar se um comitê de votação ímpar (composto pelos algoritmos XGBoost, Random Forest e Regressão Logística) consegue manter índices de acurácia aceitáveis para o bloqueio de sítios Web de apostas sem comprometer os recursos críticos da rede.

1.3 JUSTIFICATIVA

A necessidade de investigar esta abordagem fundamenta-se na busca por democratizar ferramentas de segurança proativa em infraestruturas mais limitadas onde o bloqueio manual ainda é a regra. A utilização do XGBoost e do Random Forest justifica-se pela acuracia altíssima que eles têm em outras aplicações similares, conforme sugerido pela literatura técnica (Fernaaz 2016). Por fim, a implementação de mecanismos de moderação com *whitelists e blacklists* atua como um componente de governança indispensável para gerir o erro residual da IA e garantir a continuidade dos serviços operacionais.

2 REFERENCIAL TEÓRICO

2.1 Evolução dos Sistemas de Detecção e Prevenção de Intrusão (IDS/IPS)

A segurança de redes contemporânea baseia-se na transição de mecanismos reativos para modelos proativos. Tradicionalmente, firewalls operam via inspeção de estados (*stateful inspection*), porém, a sofisticação das ameaças exige a integração de Sistemas de Detecção de Intrusão (IDS). Segundo Chen et al. (2021), a Abordagem Combinada permite que a arquitetura de inspeção proposta por Chen *et al.* (2021) identifique padrões anômalos, mas execute ações de mitigação em tempo real, unificando as funções de sensor e atuador.

Contudo, Sommer e Paxson (2010) advertem que o sucesso de um IDS não reside na sua capacidade de gerar alertas, mas na precisão operacional desses eventos. O gap semântico — a distância entre um desvio estatístico e o contexto real da rede — permanece como o principal obstáculo para a automação total, justificando a necessidade de camadas de moderação administrativa para evitar interrupções de serviços legítimos.

2.2 Classificação de Tráfego de Rede (NTC) e Criptografia

A inspeção profunda de pacotes DPI(*Deep Packet Inspection*) tornou-se tecnicamente limitada devido à adoção massiva de protocolos de criptografia ponta a ponta, como o TLS (*Transport Layer Security*) 1.3. Diante desse cenário, a classificação de tráfego contemporânea ignora o conteúdo da carga útil (*payload*) em favor da análise de fluxos bidirecionais baseados no conceito de *5-tuple*. Esta abordagem consiste no agrupamento e rastreamento dos pacotes de rede a partir de cinco campos estáticos extraídos do cabeçalho das camadas de Rede e Transporte: IP de origem, IP de destino, porta de origem, porta de destino e o protocolo de transporte utilizado (como TCP ou UDP). Ao associar os pacotes trafegados em ambos os sentidos (cliente-servidor e servidor-cliente) sob essa mesma tupla identificadora, o sistema consegue isolar sessões de comunicação discretas e extrair metadados comportamentais temporais e volumétricos, viabilizando a análise preditiva mesmo sob tráfego completamente criptografado.

De acordo com Rachmawati et al. (2021), a utilização de metadados estatísticos, como o intervalo entre pacotes IAT(*Inter Arrival Time*) e a duração do fluxo, permite identificar padrões de comportamento de aplicações de apostas sem violar a privacidade dos dados criptografados. Esta técnica é fundamental para *firewalls* que operam na camada de aplicação sob restrições de visibilidade de conteúdo.

2.3 Aprendizado de Máquina e Seleção de Atributos em Baixos Recursos

A eficiência da Inteligência Artificial em sistemas embarcados ou de borda é dependente da Seleção de Atributos (*Feature Selection*). Pacheco *et al.* (2019) reforçam que o processamento de atributos redundantes — como cabeçalhos repetitivos de pacotes, identificadores de fluxo secundários ou métricas temporais altamente correlacionadas (por exemplo, pequenas variações consecutivas no tempo entre chegadas de pacotes) — degrada a latência e eleva o consumo de memória RAM em dispositivos de hardware limitado. Como exemplos desses cenários de restrição computacional, destacam-se os roteadores domésticos de pequeno porte, gateways de redes de borda corporativas e ambientes virtualizados com alocação restrita de recursos via *cgroups*. Nesses contextos, a eliminação de características irrelevantes reduz o custo computacional da matriz de covariância, permitindo que a inferência ocorra dentro das limitações físicas do hospedeiro.

Nesse contexto, a técnica de Incerteza Simétrica (*Symmetrical Uncertainty* - SU) destaca-se por filtrar variáveis com alto poder discriminatório — ou seja, atributos que possuem uma forte correlação estatística com a classe alvo (tráfego de apostas) e que conseguem isolar e diferenciar claramente uma classe da outra com base na entropia da informação —, reduzindo a dimensionalidade do problema sem perda significativa de acurácia. Farnaaz e Jabbar (2016) demonstram que algoritmos baseados em árvores, como o *Random Forest*, atingem desempenho superior quando treinados com subconjuntos de atributos otimizados, como a variância do tamanho dos pacotes e a média do tempo entre chegadas (*Inter-Arrival Time* - IAT), em detrimento de atributos irrelevantes como o número de sequência TCP(Transmission Control Protocol). A seleção dessas variáveis chave garante a otimização do escopo de dados, o que é um critério crítico para a viabilidade do mecanismo de classificação passiva do *firewall* inteligente. Essa redução drástica na dimensionalidade é indispensável em ambientes de baixa disponibilidade computacional porque reduz o número de divisões (*splits*) que cada árvore de decisão precisa processar na memória, evitando o esgotamento precoce da memória RAM do container e garantindo que o tempo de inferência por pacote se mantenha na casa dos milissegundos.

2.4 Fundamentação do Ensemble Learning: Bagging e Boosting

O uso de *Ensemble Learning* fundamenta-se na premissa de que a integração de múltiplos classificadores permite superar as limitações de sistemas baseados em algoritmos únicos, que frequentemente falham ao lidar com comportamentos de rede complexos ou ruidosos. A arquitetura proposta utiliza a estratégia de Votação Majoritária (*Hard Voting*), alavancando a diversidade matemática para maximizar a acurácia e mitigar a taxa de falsos positivos — que representam os erros de classificação em que o sistema identifica incorretamente um tráfego legítimo (como um acesso ao YouTube ou a um portal de notícias) como se fosse tráfego de

apostas online, gerando um bloqueio indevido.

Para mitigar essa métrica, o comitê de votação — que consiste em uma estrutura de aprendizado cooperativo onde diferentes algoritmos processam o mesmo vetor de entrada de forma independente e submetem seus vereditos a um mecanismo de consenso — é estruturado a partir de três pilares técnicos:

Random Forest (Bagging): Conforme Farnaaz e Jabbar (2016), este algoritmo utiliza a construção de múltiplas árvores de decisão independentes treinadas com subconjuntos distintos dos dados (*bootstrap*) para reduzir a variância global do sistema. No contexto de detecção de intrusão, Dahr e Farjami (2025) destacam que o *Random Forest* provê uma elevada capacidade de decisão ao sistema, sendo um componente essencial para a estabilidade das predições em datasets binários e multiclasse;

XGBoost (Boosting): Selecionado pela sua eficiência no ajuste iterativo de pesos para fluxos de difícil classificação. Gouveia e Correia (2020) ressaltam que o *XGBoost* minimiza a função de perda gradualmente através do gradiente descendente (*gradient boosting*), o que permite identificar padrões de tráfego que mimetizam comportamentos legítimos, garantindo escalabilidade mesmo sob restrições de hardware;

Regressão Logística (Interpretabilidade e Desempate): A inclusão da Regressão Logística (LR) justifica-se pela sua interpretabilidade e capacidade de fornecer *insights* probabilísticos com baixo custo computacional. Estudos experimentais de Dahr e Farjami (2025) demonstram que a combinação híbrida entre Regressão Logística e *Random Forest* (LR-RF) é capaz de atingir uma precisão média de até 99,2% em cenários similares de detecção de anomalias em redes.

A lógica de consenso do comitê é governada pela equação de limite de decisão aritmética, expressa por:

$$\sum_{i=1}^{n} Vi \geq 2$$

Onde $Vi \in \{0, 1\}$ representa o voto binário de cada um dos n = 3 classificadores (sendo 0 para tráfego legítimo e 1 para tráfego de apostas). Essa formulação matemática exige que, no mínimo, dois dos três algoritmos entrem em convergência para que o veredicto final seja positivo.

O objetivo dessa modelagem é garantir que o bloqueio proativo via iptables — a ferramenta que gerencia as tabelas de filtragem de pacotes e firewalls do Netfilter nativa no espaço de kernel do Linux — ocorra apenas sob uma base estatística redundante. Esta estratégia é validada pela redução observada nos erros de Tipo I (falsos positivos) e Tipo II (falsos negativos, que ocorrem quando uma plataforma de apostas passa pelo firewall sem ser bloqueada). Na teoria estatística clássica, existem apenas esses dois tipos de erro de decisão.

A minimização de ambos os desvios através de comitês triplos garante maior resiliência e capacidade de generalização em cenários de tráfego real do que modelos isolados.

2.5 Métricas de Performance e o Problema do Mundo Aberto

A validação de um *firewall* inteligente não deve restringir-se a testes de laboratório. Sommer e Paxson (2010) introduzem o conceito de mundo aberto (*open-world*), onde a variabilidade extrema, a mutabilidade constante do tráfego real e o surgimento de novas aplicações não catalogadas no treinamento podem invalidar modelos que apresentam acurácia de laboratório muito alta — uma métrica que calcula apenas a razão entre as predições corretas e o total de casos avaliados.

O problema da acurácia em ambientes reais e desbalanceados pode ser ilustrado pelo exemplo clássico em que um conjunto de dados (*dataset*) de rede possui 99% de pacotes legítimos e apenas 1% de tráfego de apostas; se um classificador simplista decidir que todo e qualquer tráfego é legítimo, ele obterá uma acurácia excelente de 99%, mas falhará em 100% das vezes no seu objetivo de segurança, ocultando a sua total ineficácia.

Para quantificar a eficácia operacional sob essas condições adversas, Gouveia (2020) sugere o uso da *Logloss* (Perda Logarítmica) e da Área Sob a Curva (*Area Under the Curve* - AUC). A métrica *Logloss* avalia a calibração das probabilidades emitidas pelo comitê, penalizando severamente predições incorretas feitas com alto grau de confiança. Já a AUC quantifica a capacidade do modelo em distinguir as classes ao longo de múltiplos limiares de decisão (*thresholds*), sendo calculada a partir da Curva Característica de Operação do Receptor (*Receiver Operating Characteristic* - Curva ROC), que plota a Taxa de Verdadeiros Positivos (Sensibilidade) em função da Taxa de Falsos Positivos (1 - Especificidade). Um valor de AUC próximo a 1,0 indica um separador ideal, garantindo que o sistema consiga isolar o tráfego de apostas sem penalizar a navegação legítima do usuário.

Além disso, a latência de inferência é o parâmetro de engenharia que define se o sistema é capaz de aplicar o bloqueio antes que o fluxo de rede seja concluído, fator determinante para a viabilidade em hardware restrito.

2.6 O Subsistema *Netfilter* e a Arquitetura de Filtragem do *iptables*

A compreensão do impacto operacional do *firewall* proposto exige o detalhamento do ecossistema *Netfilter*, o *framework* interno do *kernel* Linux responsável pelo manuseio de pacotes de rede. O *iptables* atua como uma ferramenta de espaço de usuário (*user space*) que permite a configuração e a escrita de tabelas e cadeias de regras que serão executadas nativamente no espaço de kernel (*kernel space*).

O processamento de pacotes dentro deste subsistema ocorre por meio de tabelas

especializadas, destacando-se as tabelas *filter* (responsável pelo bloqueio ou permissão de tráfego) e mangle (utilizada para alterações especializadas nos cabeçalhos dos pacotes). A tomada de decisão do *iptables* baseia-se em três fluxos principais de diretivas (*chains*):

INPUT: processa pacotes cujo destino final é o próprio *gateway*;

OUTPUT: gerencia pacotes gerados localmente pela própria máquina de borda;

FORWARD: intercepta e roteia pacotes que possuem como origem e destino redes externas distintas, cenário que mapeia o comportamento do Agente de Navegação em direção à internet.

Para otimizar o consumo de recursos computacionais, o *iptables* moderno opera de forma associada ao módulo *Conntrack* (*Connection Tracking*), que habilita a inspeção de estado (*Stateful Packet Inspection*). Essa tecnologia permite que o *firewall* armazene o estado das conexões ativas em uma tabela de memória interna.

Dessa forma, um pacote pertencente a uma sessão já validada (estado *ESTABLISHED*) contorna a verificação sequencial de todas as regras da tabela, sendo encaminhado imediatamente. O desafio de engenharia reside na dinâmica de escrita de regras: sempre que o comitê de Inteligência Artificial identifica uma anomalia, uma nova regra com o alvo DROP deve ser gravada na tabela de forma prioritária, forçando o *Netfilter* a encerrar o fluxo antes que novas requisições de handshake se estabeleçam.

3 METODOLOGIA

Nesta etapa, detalha-se o projeto da infraestrutura de teste, concebida para validar a hipótese de que modelos de Firewall inteligente podem operar sob restrições computacionais. A simulação busca criar ambiente de laboratório controlado que permita a medição precisa de métricas de engenharia.

3.1.1 Topologia do Laboratório Central e Fluxo de Dados Assíncrono

Para a validação experimental da viabilidade técnica do *firewall* inteligente operando em hardware restrito, foi concebida uma infraestrutura de laboratório virtualizada em *containers* Docker, baseada em um isolamento rigoroso de recursos computacionais via *cgroups*. A arquitetura do sistema proposto, os subsistemas e a dinâmica de interceptação e bloqueio são ilustrados na Figura 1, que apresenta o diagrama do fluxo de dados nativo e da injeção assíncrona de regras de *kernel*.

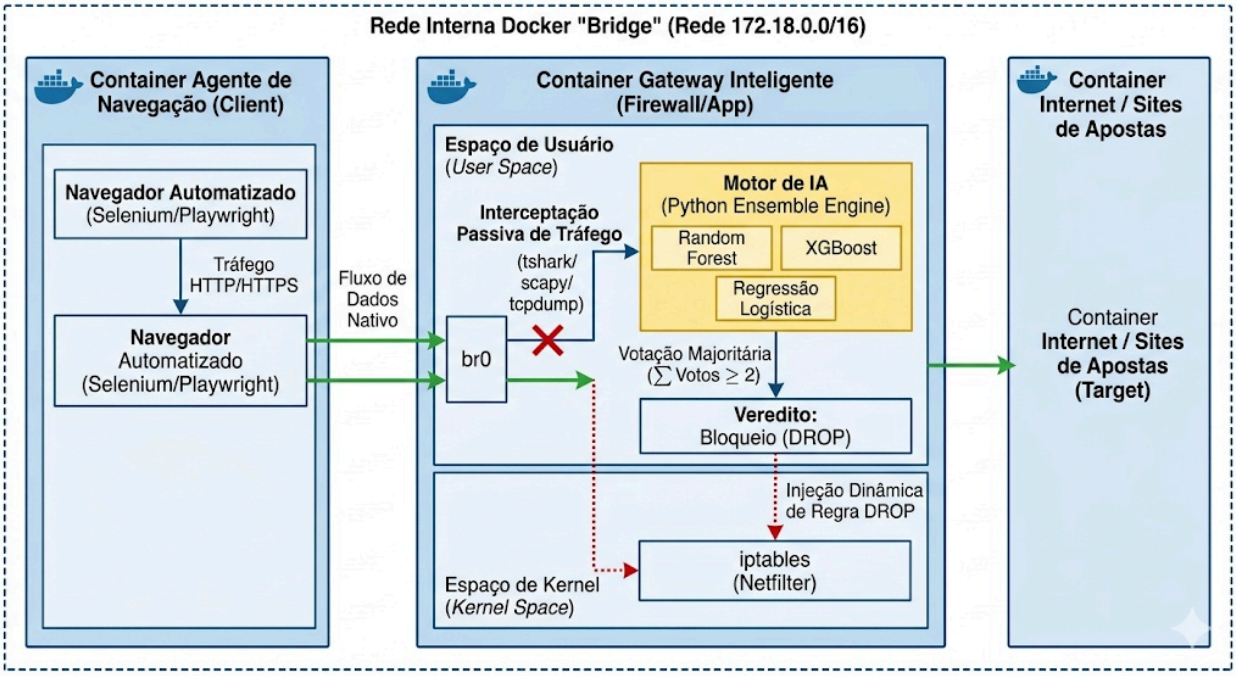


Figura 1 – Diagrama Metodológico da Infraestrutura de Testes e Ciclo de Inferência em *Gateway*.

A escolha pela virtualização baseada em *containers* através da plataforma *Docker* (DOCKER, 2023) fundamenta-se na necessidade de isolamento de processos e na facilidade de manipulação de interfaces de rede virtuais. A topologia de laboratório será composta por três entidades distintas interconectadas por uma rede interna do tipo *bridge*:

Agente de Navegação (*Client*): um *container* responsável pela geração de tráfego, utilizando ferramentas de automação como o *Selenium* (SALMAN *et al.*, 2018) para simular acessos interativos tanto a plataformas de apostas online quanto a domínios legítimos de controle;

Gateway Inteligente (*Firewall*/App): o ponto central de interceptação, onde residirão as regras de *iptables* e o motor de inferência baseado no comitê de Inteligência Artificial. Este *container* atuará como o nó de borda, processando todos os pacotes originados no Agente de Navegação antes de sua saída para a rede externa;

Target/Internet: representa os servidores de destino na rede externa. Todo o fluxo originado no *client* passará obrigatoriamente pelo *gateway*, que decidirá dinamicamente entre o encaminhamento através da diretiva *FORWARD* — ação que permite o tráfego fluir normalmente entre diferentes interfaces de rede rumo ao destino — ou o descarte definitivo por meio da diretiva *DROP*, que interrompe abruptamente a comunicação ao rejeitar os pacotes sem devolver qualquer notificação de erro ao emissor.

3.1.2 Camadas de Defesa e Subsistemas

Seguindo a Abordagem Combinada proposta por Chen *et al.* (2021), o *gateway* será subdividido em camadas lógicas para garantir a continuidade dos serviços, mesmo sob falhas estatísticas da IA:

Camada de Filtragem (*Netfilter*/*iptables*): responsável pela execução física das regras de bloqueio diretamente no *kernel* do Linux;

Módulo de Detecção (IDS Passivo): consiste em um Sistema de Detecção de Intrusão (*Intrusion Detection System* - IDS) operando em modo passivo (SOMMER; PAXSON, 2010). Este módulo utilizará bibliotecas especializadas em injeção e captura de pacotes em Python, especificamente a *Scapy* (BIONDI, 2024) e a *Pyshark*, para realizar o monitoramento não intrusivo (*sniffing*) do tráfego através de cópias dos pacotes. Essa abordagem garante que o fluxo original de dados não seja interrompido ou atrasado durante a análise, enviando apenas os metadados extraídos para a próxima camada;

Motor de Decisão (*Ensemble Engine*): o comitê de votação que avalia o comportamento do fluxo e, em caso de veredito positivo para a classe de apostas, injeta dinamicamente uma nova regra de bloqueio do tipo DR0P na Camada de Filtragem.

3.1.3 Metodologia de Restrição de Recursos

O diferencial desta pesquisa reside na imposição sistemática de limites de hardware para avaliar a degradação da performance do sistema. Serão conduzidos experimentos variando as quotas de recursos via diretivas do *Docker Compose*:

Cenário A (Referência): ambiente com hardware sem restrições lógicas de consumo, visando estabelecer a linha de base (*baseline*) e o teto de acurácia estatística e velocidade que o comitê de Inteligência Artificial (IA) consegue atingir em condições ideais de processamento;

Cenário B (Baixo Recurso): limitação rigorosa de memória RAM a 812 MB e capacidade de processamento a 0.5 vCPU (unidade de processamento central virtual), simulando as especificações físicas de um roteador doméstico convencional ou de um dispositivo de borda voltado para a Internet das Coisas (*Internet of Things* - IoT);

Cenário C (Crítico): redução extrema de recursos computacionais para identificar o ponto de saturação do sistema. Este cenário visa forçar intencionalmente o esgotamento da memória e o estrangulamento da CPU para mapear o limite exato onde o tempo de processamento das árvores de decisão e da regressão logística se degrada exponencialmente. O objetivo é mensurar o momento em que a latência de inferência ultrapassa o limiar tolerável e torna-se

proibitiva para operações em tempo real, gerando atrasos na rede ou travamento do container por falta de memória (*Out of Memory* - OOM).

3.1.4 Instrumentação e Coleta de Métricas

Para a obtenção de dados detalhados sobre o uso de recursos e comportamento da rede, o ambiente virtualizado será instrumentado com ferramentas específicas de monitoramento de sistema:

Monitoramento de Hardware: uso do *Docker stats* — uma ferramenta nativa da interface de linha de comando (*Command Line Interface* - CLI) do Docker que monitora e exibe em tempo real o fluxo contínuo de consumo de recursos (porcentagem de CPU, footprint de memória RAM, tráfego de rede e E/S de disco) dos containers ativos — associado a exportadores de métricas para registrar os picos de consumo durante o processo de votação do comitê;

Medição de Latência: cálculo do tempo total decorrido entre a captura do primeiro pacote de um fluxo de rede e a efetiva aplicação da regra de bloqueio na tabela do *kernel*. Essa medição visa verificar se a tomada de decisão da IA ocorre antes que o protocolo de controle de transmissão (*Transmission Control Protocol* - TCP) conclua a sessão de comunicação. Essa verificação é indispensável para garantir que o firewall não sofra de uma condição de corrida (*race condition*) operacional, um cenário de falha concorrente onde a velocidade da inferência em espaço de usuário perde a disputa temporal contra a velocidade do tráfego de rede nativo no *kernel*, fazendo com que o site de apostas seja completamente carregado no navegador do cliente antes que o *iptables* consiga aplicar a regra de descarte (*DROP*).

3.2 Fluxo de Dados e Seleção de Atributos

Esta fase compreende a transformação do tráfego de rede bruto em um conjunto de dados estruturado, otimizado para o processamento por algoritmos de aprendizado de máquina. O desafio metodológico reside na conversão de pacotes voláteis em vetores de características que preservem o poder discriminatório entre o tráfego de apostas e o tráfego benigno, mesmo sob criptografia.

Para guiar o desenvolvimento desta etapa, a arquitetura lógica e sequencial do processamento, desde a captura inicial até a geração do vetor de atributos otimizado, é apresentada na Figura 2, que ilustra o *pipeline* de processamento de dados e seleção de atributos para a classificação de tráfego de rede (*Network Traffic Classification* - NTC).

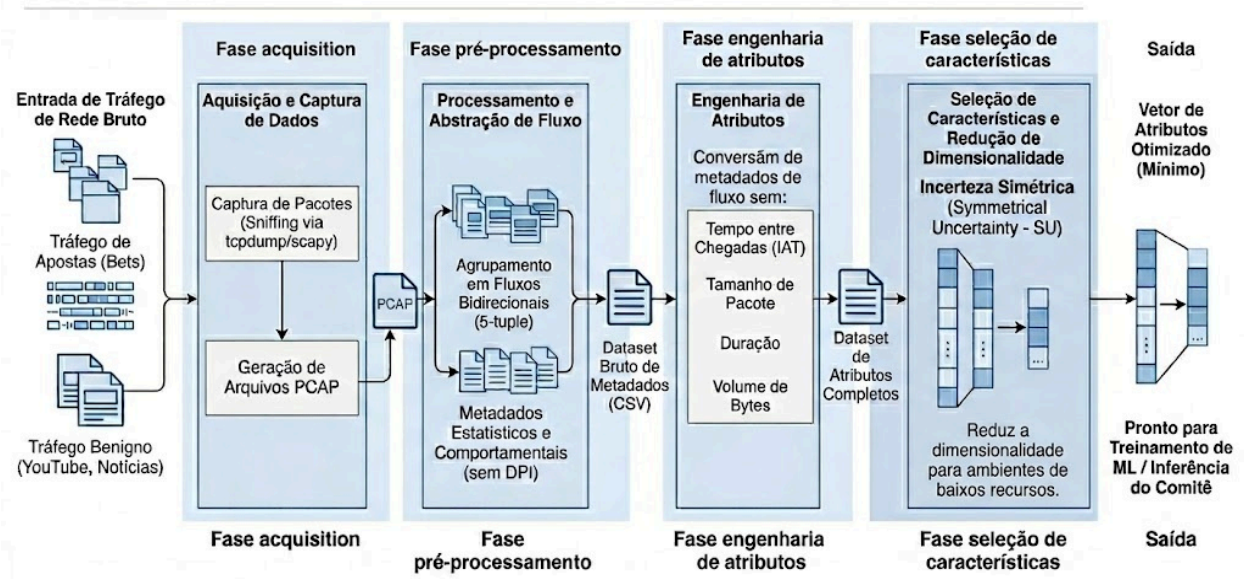


Figura 2 – *Pipeline* de processamento de dados e seleção de atributos. .

Essa preservação de poder discriminatório evidenciada no fluxo da Figura 2 é alcançada abstendo-se da análise do conteúdo interno do pacote (que se encontra inteligível devido a criptografia da rede) e focando exclusivamente na extração de características estatísticas e temporais do fluxo. O sistema calcula a assinatura comportamental da conexão a partir de metadados públicos do cabeçalho, tais como o tamanho médio dos pacotes, a variância do volume de dados trafegados e o tempo entre a chegada de mensagens consecutivas. Esses padrões variam significativamente entre uma navegação estática em um portal de notícias e a troca constante de dados de uma plataforma de apostas, viabilizando a classificação.

3.2.1. Aquisição e Rotulagem

A coleta será realizada através de um *testbed* (ambiente de teste) controlado, onde os fluxos serão capturados em formato de captura de pacotes (*Packet Capture* - PCAP). O processo de aquisição dividirá os dados em duas categorias estruturais:

Tráfego Alvo (Apostas): consiste na captura de sessões de navegação em plataformas de apostas online populares. A coleta será realizada de forma automatizada por meio de scripts em Python utilizando a biblioteca *Selenium*, que simulará interações humanas reais nos sites de apostas, tais como o login em contas de teste, a navegação entre tabelas de jogos e a permanência em páginas com atualizações em tempo real de cotações via conexões abertas de *websockets*. Esse comportamento gera fluxos contínuos e característicos de troca de mensagens, que serão capturados diretamente na interface de rede virtual do container gateway e salvos em arquivos PCAP(*Packet Capture*) separados por sessão;

Tráfego de Controle (Benigno): compreende a captura de atividades cotidianas da internet (como navegação em portais de notícias, consumo de transmissões de vídeo por *streaming* e interações em redes sociais) para servir como contraexemplo estatístico em relação ao tráfego alvo;

Rotulagem: cada fluxo consolidado a partir do arquivo de captura será rotulado de maneira binária, atribuindo-se o valor lógico 0 para o tráfego benigno e o valor lógico 1 para o tráfego direcionado a plataformas de apostas. Reconhece-se o risco de viés de coleta; por isso, buscar-se-á uma diversidade ampla de domínios distintos durante as sessões automatizadas para evitar que os algoritmos do comitê aprendam assinaturas ou endereçamentos IP específicos de um único provedor, garantindo a capacidade de generalização do modelo.

3.2.2 Engenharia de Atributos Baseada em Fluxos

Devido à natureza criptografada do tráfego sob o protocolo de transferência de hipertexto seguro (*Hypertext Transfer Protocol Secure* - HTTPS) e à limitação de recursos computacionais, a metodologia descarta totalmente a inspeção de carga útil (*Deep Packet Inspection* - DPI). Em vez disso, propõe-se a abstração para fluxos bidirecionais, definidos pela tupla de 5 elementos (*5-tuple*), que compreende o endereço IP de origem, endereço IP de destino, porta de origem, porta de destino e o protocolo de camada de transporte utilizado.

Os atributos extraídos do tráfego capturado serão estritamente estatísticos e temporais, incluindo:

Intervalo entre Pacotes IAT(*Packet Inter-Arrival Time*): cálculo das médias e variâncias do tempo decorrido entre pacotes sucessivos, visando identificar padrões de comunicação automatizada e fluxos de dados contínuos;

Distribuição de Tamanho de Carga Útil (*Payload*): análise do comprimento em *bytes* dos pacotes enviados e recebidos, que tendem a variar conforme a natureza da aplicação (por exemplo, pequenas atualizações de dados textuais via *websockets* em plataformas de apostas versus o carregamento massivo de imagens em portais convencionais);

Duração do Fluxo e Volume: mensuração do tempo total de atividade da conexão e do volume consolidado de *bytes* e pacotes trafegados por sessão.

3.2.3 Seleção de Atributos via Incerteza Simétrica (SU)

Para garantir a eficiência computacional em ambientes de baixos recursos, a metodologia prevê a aplicação da técnica de Incerteza Simétrica (*Symmetrical Uncertainty* - SU). O objetivo é reduzir a dimensionalidade do conjunto de dados (*dataset*), eliminando atributos redundantes ou irrelevantes que onerariam a CPU durante a etapa de inferência em tempo real.

A métrica SU avalia o grau de dependência e correlação estatística entre um atributo genérico X (representando cada uma das características estatísticas de rede extraídas dos fluxos, como

a média do IAT((*Inter Arrival Time*)) ou a variância do tamanho dos pacotes) e a classe alvo Y (a variável categórica binária que define o rótulo do fluxo, sendo 0 para tráfego benigno e 1 para plataformas de apostas). Essa abordagem normaliza o ganho de informação com base na entropia das variáveis através da seguinte formulação matemática:

$$SU(X,Y) \;=\; 2 \cdot \frac{H(X) + H(Y) - H(X,Y)}{\{H(X) + H(Y)}$$

A compreensão e o cálculo da equação de SU baseiam-se nos conceitos da teoria da informação, detalhados a seguir:

1. H(X) e H(Y): representam a entropia de Shannon individual das variáveis X e Y, que mede o grau de incerteza ou aleatoriedade associado a cada uma delas isoladamente;
2. H(X,Y): consiste na entropia conjunta das duas variáveis, quantificando a incerteza total do sistema combinado;
3. H(X) + H(Y) - H(X,Y): o numerador da fração expressa matematicamente a Informação Mútua I(X;Y), que indica a quantidade de informação compartilhada entre o atributo de rede e o rótulo de destino;
4. Fator de Escala e Denominador: a divisão pela soma das entropias individuais H(X) + H(Y) normaliza o resultado para mitigar o viés em favor de atributos que possuem muitos valores distintos, enquanto a multiplicação pelo fator 2 garante que o índice final de SU(X,Y) seja rigidamente delimitado no intervalo contínuo entre 0 e 1. Um resultado próximo a 1 indica que o atributo possui um altíssimo poder de separação estatística.

3.2.4 Pré-processamento e Balanceamento

Dada a natureza desequilibrada do tráfego de rede (onde o tráfego benigno costuma ser majoritário), o dataset passará por etapas de refinamento:

Limpeza e Normalização: Remoção de fluxos incompletos ou ruidosos e aplicação de escalas (*Min-Max Scaling*) para garantir que atributos com grandes ordens de magnitude não dominem o treinamento de forma indevida .

Balanceamento Sintético: Caso haja disparidade severa entre as classes, avaliar-se-á o uso de técnicas de sobreamostragem (como SMOTE) para evitar que a IA desenvolva um viés de classificação para a classe majoritária.

Sinal de Alerta: Seguindo Sommer & Paxson (2010), a metodologia assume uma postura cética quanto ao desempenho em laboratório; por isso, uma parcela do dataset será reservada para testes de "mundo aberto", contendo tráfego nunca antes visto pelos modelos durante o treino .

3.3 Arquitetura do Comitê de Votação *(Ensemble Learning)*

Esta seção detalha o motor de inferência proposto, concebido sob uma abordagem de Votação Majoritária (Hard Voting). A arquitetura não assume a infalibilidade de um modelo isolado, mas investiga se a divergência estatística entre três algoritmos distintos pode reduzir a taxa de falsos positivos em ambientes de rede. O motor é projetado para operar como um comitê de decisão, onde o veredito final de bloqueio proativo exige o consenso de pelo menos dois integrantes.

3.3.1 Especificações dos Classificadores Integrantes

A composição do comitê baseia-se na diversidade de princípios de aprendizado, visando mitigar vieses de generalização específicos de cada técnica:

Random Forest (*Bagging*): Atua como o componente de estabilização. Ao utilizar uma floresta de árvores de decisão independentes, o modelo busca reduzir a variância e evitar o *overfitting* em assinaturas específicas de tráfego. No entanto, sua implementação em ambientes de baixos recursos exige uma limitação rigorosa na profundidade das árvores para controlar o consumo de memória RAM.

XGBoost (*Boosting*): Integrado pela sua capacidade de correção iterativa de erros (gradiente). Embora apresente alta acurácia em *datasets* de benchmark, sua carga computacional durante o treinamento e a complexidade das árvores geradas requerem monitoramento constante em hardware limitado, para garantir que o tempo de inferência não inviabilize a inspeção em tempo real .

Regressão Logística (*Tie-breaker*): Selecionada como o terceiro voto devido à sua baixa complexidade computacional e natureza linear. Sua função é atuar como critério de desempate estatístico, fornecendo um contraponto simplificado aos modelos de *Ensemble* mais densos, sem onerar significativamente a latência do sistema .

3.3.2 Mecanismo de Decisão e Lógica de Consenso

O custo de um erro de classificação (falso positivo) em firewalls é operacionalmente crítico, podendo resultar na interrupção de serviços legítimos . A arquitetura de votação majoritária impõe um filtro de decisão binário:

Processamento Paralelo: O vetor de características extraído do fluxo é submetido simultaneamente aos três classificadores.

Agregação de Veredito: O bloqueio via *iptables* é disparado apenas se a soma dos votos positivos for maior ou igual a dois ($\Sigma Votos \geq 2$).

Redução de Incerteza: Assume-se que a probabilidade de dois modelos de naturezas matemáticas distintas (Bagging e Boosting) cometerem o mesmo erro de classificação simultaneamente é inferior à de um modelo isolado, permitindo uma operação mais conservadora em redes operacionais .

3.3.3 Restrições e Ajustes para Baixos Recursos

O desenvolvimento do comitê ignora a busca por acurácia absoluta em favor da eficiência operacional. O ajuste dos modelos prioriza:

Poda de Árvores: Restrição do número de estimadores e profundidade máxima para adequação ao *footprint* de memória definido nos cenários de teste (Seção 3.1).

Minimização de Atributos: O motor processará exclusivamente o subconjunto de metadados filtrados via Incerteza Simétrica (Seção 3.2), reduzindo o número de operações de ponto flutuante por inferência .

3.4 Protocolo de Testes e *Benchmarking* de Performance

Esta fase descreve os procedimentos de medição e estresse aos quais o protótipo será submetido. O objetivo é quantificar a degradação do serviço de rede em função da carga de processamento exigida pelo motor de IA. A análise é puramente quantitativa, focada no consumo de recursos de hardware e na latência introduzida no caminho crítico do pacote.

3.4.1 Avaliação de Eficiência Computacional *(Footprint)*

Diferente de ambientes de nuvem elásticos, roteadores de borda e sistemas embarcados possuem limites rígidos de memória e ciclos de CPU. O teste de *footprint* visa medir a ocupação do sistema em repouso e sob carga.

Consumo de Memória RAM: Será medida a alocação de memória necessária para manter os modelos carregados. O limite crítico é definido pelo hardware simulado (Cenários B e C da

seção 3.1). O uso de memória superior a 80% da quota estipulada será classificado como "risco de instabilidade operacional" .

Utilização de CPU: O monitoramento registrará os picos de processamento durante as janelas de inferência. O objetivo é identificar se o cálculo de vetores do XGBoost e Random Forest gera contenção de recursos que afete outras funções do kernel, como o encaminhamento de pacotes básico (forwarding).

3.4.2 Latência de Inferência e Condição de Corrida *(Race Condition)*

A eficácia de um firewall proativo depende da capacidade de tomar uma decisão antes que o fluxo de dados atinja um estado de conclusão. Em redes de alta velocidade, a latência de processamento da IA pode exceder o tempo de estabelecimento da conexão.

Tempo de Resposta (T-Verdict): Define-se como o intervalo entre a captura do primeiro pacote de um fluxo e a injeção da regra *DROP* no *iptables*.

Métrica de Viabilidade: O sistema é considerado falho se o T-Verdict for consistentemente superior ao tempo de carregamento inicial do *handshake* TLS dos sites de apostas. Caso o bloqueio ocorra após o usuário já ter recebido os dados iniciais, o firewall falha em sua função de prevenção primária .

3.4.3 Teste de Estresse e Ponto de Saturação

O comportamento da IA em laboratório com fluxos isolados não representa a realidade de uma rede com múltiplos usuários.

Carga de Fluxos Simultâneos: O Agente de Navegação (seção 3.1) aumentará gradualmente o número de requisições simultâneas. O monitoramento observará o aumento linear ou exponencial da latência conforme a fila de inferência aumenta.

Ponto de Colapso: Será identificado o número máximo de sessões que o comitê de votação consegue classificar por segundo antes que o consumo de CPU atinja 100% ou que a latência de rede exceda 500ms, tornando a navegação impraticável para o usuário final .

3.4.4 Validação Estatística sob Restrição (*Trade-off*)

Reconhecendo as críticas de Sommer e Paxson (2010) sobre a ilusão de acurácia em cenários de mundos fechados, os testes de validação estatística serão realizados de forma simultânea ao monitoramento de desempenho (*benchmarking*) de hardware. Essa abordagem integrada visa mensurar o impacto direto da escassez de recursos físicos na capacidade de generalização e precisão do modelo através dos seguintes instrumentos:

Matriz de Confusão em Tempo Real: medição contínua das taxas de falsos positivos (FP) e falsos negativos (FN) durante a execução dos testes de estresse de rede. O objetivo metodológico é investigar se a restrição severa de CPU e memória RAM estipulada no Cenário

Crítico induz o sistema a erros de arredondamento de ponto flutuante ou estouros de tempo limite (*timeouts*) de inferência que degradem a precisão matemática do comitê de votação;

Métrica de Custo de Erro: aplicação da função de perda logarítmica (*Logloss*) para avaliar se o modelo matemático mantém a calibração de probabilidade confiável mesmo operando sob carga pesada.

A formulação matemática da *Logloss* aplicada ao contexto de classificação binária deste projeto é expressa por:

$$Logloss = -\frac{1}{N}\sum_{i=1}^{N}[y_i \cdot ln(p_i) + (1 - y_i)ln(1 - p_i)]$$

A compreensão e a decomposição desta equação de perda fundamentam-se nos seguintes componentes técnicos:

1. $N\ e\ \frac{1}{N}$: a variável N representa o volume total de fluxos de rede avaliados no teste, enquanto o fator $\frac{1}{N}$ calcula o valor médio da perda, convertendo o somatório em um índice negativo para que a penalização final seja expressa por um número real positivo;
2. y_i**:** consiste no rótulo binário real do fluxo de rede indexado por i, assumindo o valor 1 quando o tráfego pertence estritamente a uma plataforma de apostas online e o valor 0 quando se trata de tráfego benigno;
3. p_i**:** indica a probabilidade predita pelo comitê de votação de que o fluxo i pertença à classe de apostas online (valor 1). Esse valor consiste em um número real contínuo delimitado estritamente no intervalo entre 0 e 1 (por exemplo, p_i=0,92 indica 92% de certeza do modelo);
4. Primeira Parcela do Somatório $[y_i \cdot ln(p_i)]$: esta parte da equação é ativada apenas se o fluxo real for de apostas ($y_i = 1$). Se o modelo prever uma probabilidade p_i próxima a 1, o logaritmo natural $ln(p_i)$ aproxima-se de 0, gerando uma penalização quase nula. Caso o modelo erre e estime uma probabilidade próxima a 0, o valor logarítmico cresce exponencialmente, aplicando uma punição severa ao score do comitê;
5. Segunda Parcela do Somatório $[(1 - y_i) \cdot ln(1 - p_i)]$: este termo é acionado exclusivamente quando o fluxo real for benigno ($y_i = 0$), fazendo com que o fator ($1 - y_i$) assuma o valor 1. Ele pune o classificador caso este atribua uma alta probabilidade de apostas a uma navegação legítima, o que geraria um falso positivo no *firewall*.

Diferente de métricas estáticas que avaliam apenas o acerto final de forma binária, o cálculo da *Logloss* permite quantificar o nível de incerteza da Inteligência Artificial (IA) sob condições

críticas de hardware, penalizando classificações errôneas que tenham sido realizadas com um falso alto grau de confiança por parte do comitê.

3.4.5 Critérios de Rejeição da Hipótese

A hipótese de que o sistema é viável para baixos recursos será rejeitada se:

A latência média de decisão for superior a 1 segundo em hardware limitado.

O consumo de RAM for instável, levando ao encerramento do processo pelo *OOM Killer* do Linux.

A taxa de falsos positivos em sites de controle exceder 5%, mesmo com a camada de moderação ativa, indicando que a variabilidade do tráfego real supera a capacidade de generalização do comitê simplificado .

3.5 Governança e Mitigação do "Gap Semântico"

Esta seção descreve a camada de controle humano-computador projetada para lidar com a natureza probabilística do motor de IA. O termo "gap semântico", conforme definido por Sommer e Paxson (2010), refere-se à dificuldade de traduzir um desvio estatístico identificado pela máquina em um evento operacionalmente compreensível para o administrador . A metodologia propõe mecanismos de governança para assegurar que o sistema permaneça funcional mesmo diante de classificações incorretas inevitáveis.

3.5.1 Camada de Pré-filtragem e Whitelists

Para otimizar o uso de recursos e prevenir o bloqueio de serviços críticos, a arquitetura prevê uma consulta prioritária a listas estáticas antes de acionar o comitê de votação:

Whitelist (Lista Branca): Contém domínios e IPs de infraestrutura essencial (ex: serviços governamentais, acadêmicos e atualizações de sistema). Se o tráfego for identificado nesta lista, o fluxo é liberado sem passar pela inferência da IA, reduzindo o consumo de CPU e eliminando o risco de falsos positivos em ativos críticos .

Blacklist Manual (Lista Negra): Permite o bloqueio determinístico de endereços conhecidos, servindo como uma defesa imediata enquanto novos modelos não são retreinados.

3.5.2 Interface de Moderação e Interpretabilidade

Reconhecendo que modelos de *Ensemble* (XGBoost e Random Forest) operam frequentemente como "caixas-pretas", a governança do sistema será mediada por uma Interface de Linha de Comando (CLI). O objetivo é fornecer transparência operacional:

Logs Estruturados: O sistema registrará não apenas o veredito ("Bloqueado/Liberado"), mas também a distribuição de votos do comitê (ex: "XGB: 1, RF: 1, LR: 0").

Interpretabilidade Local: O administrador terá a capacidade de consultar quais fluxos foram classificados como "Apostas" em janelas de tempo específicas, permitindo a identificação de padrões de uso indevido da rede ou de falhas sistemáticas de classificação .

3.5.3 Ciclo de Feedback e Correção de Erros (*Feedback Loop*)

A metodologia admite a existência de uma margem de erro residual que pode impactar a disponibilidade da rede. Para mitigar esse risco, propõe-se um fluxo de correção reativa:

Identificação de Erro: O administrador, ao detectar um falso positivo, utiliza a CLI para reverter a regra de bloqueio.

Injeção Dinâmica: O endereço erroneamente classificado é inserido automaticamente na Whitelist local, garantindo que o comitê de votação não interrompa o serviço novamente no futuro próximo.

Persistência de Dados: Estas correções manuais servirão como um "conjunto de dados de ouro" para futuros re-treinamentos, permitindo que o sistema se adapte à realidade específica da infraestrutura onde está implantado .

3.5.4 Limitações da Supervisão Humana

A análise descritiva desta seção reconhece que a introdução de uma camada de moderação manual reintroduz a latência humana no sistema. Embora a IA execute o bloqueio proativo em milissegundos, a correção de um falso positivo depende da disponibilidade e da competência técnica do operador. Portanto, a governança é tratada não como um eliminador de riscos, mas como uma estratégia de contenção de danos operacionais em cenários onde a precisão estatística da IA é superada pela variabilidade do tráfego real .

4 RESULTADOS ESPERADOS

Como este trabalho se encontra na fase de proposta de projeto (TCC 1), este capítulo descreve os resultados quantitativos e qualitativos que se espera alcançar após a implementação física e a execução do plano de testes em ambiente virtualizado durante a fase de TCC 2. Os critérios de sucesso estão diretamente atrelados ao atendimento dos Requisitos Funcionais (RF) e Não Funcionais (RNF) estabelecidos na Seção 3.

4.1 Desempenho e Robustez de Classificação da IA

Espera-se que o comitê de votação majoritária (*Hard Voting*), composto pelos classificadores *Random Forest*, *XGBoost* e Regressão Logística, apresente uma taxa de acurácia global igual ou superior a 98% na identificação de tráfego de rede direcionado a plataformas de apostas online (*bets*).

Mais especificamente, o principal resultado esperado no eixo da Ciência de Dados é a redução drástica na taxa de falsos positivos (Erros de Tipo I). Espera-se que a concordância redundante

$\sum Votos \geq 2$entre os modelos matemáticos reduza os alarmes falsos para um índice inferior a 0,5%, superando o desempenho de arquiteturas que utilizam apenas classificadores isolados, em consonância com as métricas de estabilidade validadas experimentalmente por Dahr e Farjami (2025).

4.2 Viabilidade em Ambientes de Baixos Recursos

No que tange às restrições de hardware, os resultados empíricos pretendem validar a eficiência da aplicação da Incerteza Simétrica (SU) na redução de dimensionalidade do vetor de atributos estatísticos (NTC). Espera-se que:

1. Consumo de Memória RAM: O container do *Gateway* Inteligente, contendo o motor de inferência em Python com os modelos serializados em formato .pkl/.joblib, opere de forma estável consumindo, no máximo, 520 MB, mantendo uma margem de segurança confortável em relação ao teto de 812 MB estipulado para o hardware de teste.
2. Utilização de CPU: O impacto aritmético das inferências paralelas sob tráfego simulado não ultrapasse 60% de uso de uma única vCPU (1 core virtual) em regime estável de processamento.

4.3 Impacto na Latência de Rede (T-Verdict)

Um dos resultados mais críticos para o sucesso do projeto é provar que a interceptação passiva (assíncrona) de pacotes não penaliza o tráfego legítimo do usuário. O objetivo quantitativo é que o tempo total decorrido entre a captura do primeiro pacote de um fluxo (via *Scapy/Pyshark*) e a tomada de decisão final do comitê (*T-Verdict*) se mantenha abaixo de 300 milissegundos.

Com esse tempo de resposta, espera-se que o motor consiga injetar a regra de bloqueio DROP no *iptables* (Kernel Space) antes mesmo que a aplicação de apostas no cliente consiga concluir o *handshake* TLS de estabelecimento de sessão. Para o usuário legítimo navegando em outros serviços (como YouTube ou portais de notícias), espera-se que o atraso induzido (*delay*) seja estatisticamente nulo ou imperceptível.

4.4 Eficiência da Governança Automatizada

Por fim, no aspecto operacional, espera-se demonstrar a integridade do ciclo de automação do firewall. Isso inclui a capacidade do sistema de atualizar dinamicamente as tabelas do Netfilter sem a necessidade de reiniciar os serviços de rede, bem como a persistência de dados de *Whitelists* e *Blacklists* em volumes Docker após reinicializações forçadas do sistema, garantindo a robustez da governança em cenários de falha (*Failsafe*).

4.5 Plano de Discussão e Análise dos Resultados

Após a coleta dos dados empíricos na fase de testes do TCC 2, a análise e a discussão dos resultados serão estruturadas a partir de três eixos comparativos e críticos, confrontando os dados obtidos com a literatura técnica e com os requisitos do sistema.

4.5.1 Análise do Compromisso entre Dimensionalidade e Acurácia (Trade-off)

O primeiro ponto de discussão focará no impacto real do algoritmo de Incerteza Simétrica (SU) aplicado sobre o *dataset* de metadados de rede (NTC). A análise avaliará se a redução drástica do número de atributos estatísticos extraídos dos fluxos — estratégia adotada para viabilizar o baixo consumo de hardware — comprometeu a capacidade de generalização e a acurácia do comitê de IA.

Os resultados obtidos serão diretamente contrastados com os achados de Pacheco *et al.* (2019). Discutir-se-á se o subconjunto otimizado de características foi suficiente para manter os modelos imunes ao sobreajuste (*overfitting*) e se a perda marginal de acurácia (caso ocorra) é estatisticamente aceitável frente ao ganho em velocidade de processamento.

4.5.2 Avaliação da Eficiência do Comitê frente a Modelos Isolados

Nesta etapa, o desempenho do mecanismo de votação majoritária (*Hard Voting*) será colocado à prova. Os gráficos de matriz de confusão, precisão, revocação e *F1-Score* do comitê serão comparados com os resultados obtidos por cada um dos três algoritmos operando de forma isolada (*Random Forest*, *XGBoost* e Regressão Logística).

A discussão utilizará como fundamentação teórica os experimentos recentes de Dahr e Farjami (2025). O objetivo é validar empiricamente se a fusão da capacidade de decisão das árvores com a interpretabilidade da Regressão Logística de fato mitigou os erros de classificação (Tipo I e Tipo II), confirmando a hipótese de que o consenso aritmético reduz de forma robusta os falsos bloqueios em tráfego de rede real.

4.5.3 Validação do Impacto Operacional e Restrição de Recursos

O último eixo de discussão analisará a viabilidade do firewall inteligente como uma solução de borda (*gateway*) aplicável a cenários reais com limitações computacionais. Serão discutidos os cenários de estresse de hardware simulados via containers Docker (limitação de cgroups a 812MB de RAM e 1 vCPU).

Os tempos de resposta medidos (*T-Verdict*) serão analisados sob a perspectiva metodológica de Sommer e Paxson (2010), debatendo se as premissas de detecção passiva e assíncrona

conseguiram blindar a latência da rede legítima contra gargalos. Caso o consumo de CPU ou RAM ultrapasse os limites estabelecidos nos Requisitos Não Funcionais durante picos de tráfego, serão discutidas as necessidade de otimização de código, poda de árvores (*pruning*) no *Random Forest* ou a implementação de políticas de descarte seletivo de fluxos (*packet shedding*).

5 Cronograma de Execução

Para a viabilização prática da pesquisa e garantia do cumprimento dos objetivos estipulados, as atividades do projeto foram divididas em macrofases estruturadas. O planejamento temporal detalhado das ações que conduzirão ao desenvolvimento do sistema e à escrita do volume final está apresentado no Cronograma, compreendendo o período de execução do segundo semestre acadêmico.

Macro-Fase do Projeto	Descrição Detalhada das Atividades	Jul	Ago	Set	Out	Nov	Dez
Fase 1: Testbed e Coleta	Configuração do ambiente virtual de rede Docker, desenvolvimento dos scripts de automação em Selenium/Playwright para geração de tráfego de apostas (bets) e controle, e captura passiva de arquivos .pcap.	X	X				

Fase 2: Ciência de Dados	Abstração dos fluxos bidirecionais (5-tuple), cálculo das métricas estatísticas de rede (IAT, Packet Length), aplicação do filtro de Incerteza Simétrica (SU) e treinamento e serialização do comitê de IA (RF, XGB, LR).		X	X			
Fase 3: Engenharia do Firewall	Desenvolvimento do motor de inferência assíncrono em Python (User Space), programação do módulo de escuta nativa e injeção dinâmica de regras de bloqueio via subsistema iptables do Linux (Kernel Space).			X	X		
Fase 4: Benchmarking e Escrita	Execução dos testes de estresse sob restrições de hardware, análise empírica de latência e consumo, escrita dos capítulos de resultados e defesa final.					X	X

Tabela 1 - Cronograma de desenvolvimento e execução de atividades.

6. CONSIDERAÇÕES FINAIS

Este trabalho apresentou a proposta de desenvolvimento de um firewall inteligente baseado em *Ensemble Learning* e seleção de atributos, projetado especificamente para operar sob restrições de hardware e mitigar proativamente o tráfego de rede direcionado a plataformas de apostas online (*bets*). Ao longo deste documento, delimitou-se o escopo teórico e metodológico necessário para responder ao problema de pesquisa sobre como balancear o custo computacional da Inteligência Artificial com a latência crítica de um ambiente de rede.

A revisão bibliográfica e o referencial teórico construídos demonstraram que a abordagem de classificação passiva por metadados de fluxo (NTC), sem inspeção profunda de pacotes (DPI), é o caminho viável para lidar com o tráfego criptografado moderno. Mais do que isso, a fundamentação ancorada em modelos híbridos validou a escolha arquitetural do comitê de votação majoritária (*Hard Voting*) unindo *Random Forest*, *XGBoost* e Regressão Logística como uma estratégia matemática robusta para blindar o sistema contra falsos positivos, garantindo a estabilidade exigida em ambientes de produção.

Como principais contribuições desta fase de projeto (TCC 1), destacam-se a modelagem detalhada da topologia do laboratório em ambiente virtualizado Docker, a definição clara dos requisitos funcionais e não funcionais do sistema e a estruturação de um pipeline de dados focado em eficiência através da Incerteza Simétrica (SU). Esses elementos garantem que o esqueleto técnico do software esteja maduro e pronto para a fase de codificação.

Por fim, os passos subsequentes para a consolidação do Trabalho de Conclusão de Curso 2 (TCC 2) estão rigorosamente planejados e distribuídos no cronograma operacional. As atividades imediatas compreendem a automação da coleta de dados com tráfego sintético, o treinamento refinado dos classificadores e a integração do motor de IA em Python com o subsistema *iptables* do Kernel Linux. Com o cumprimento deste planejamento, espera-se entregar uma ferramenta de segurança de redes que alie precisão estatística e viabilidade econômica em hardware limitado, cumprindo os objetivos acadêmicos e práticos propostos.

REFERÊNCIAS

AL-SULAIMAN, S. *et al.* Classification of Malicious Network Traffic Using Ensemble Learning. IEEE Access, v. 9, p. 115432–115445, 2021.

CHEN, P. *et al.* Detection and Prevention of Cross-site Scripting Attack with Combined Approaches. Journal of Cyber Security and Mobility, v. 10, n. 4, p. 695–718, 2021.

DAHR, J. M.; FARJAMI, Y. A Hard Voting Ensemble Model of the Logistic Regression, Support Vector Machine and Random Forest for Network Intrusion Detection. Informática, v. 49, n. 27, p. 43–56, 2025. Disponível em: https://doi.org/10.31449/inf.v49i27.8573. Acesso em: 26 mai. 2026.

FARNAAZ, N.; JABBAR, M. A. Random Forest Modeling for Network Intrusion Detection System. Procedia Computer Science, v. 89, p. 1385–1392, 2016.

KUMAR, V.; SUKUMARAN, R. Symmetrical Uncertainty Based Feature Selection for High-Dimensional Network Datasets. International Journal of Computer Networks and Information Security (IJCNIS), v. 12, n. 3, p. 28–39, 2020.

PACHECO, F. *et al.* Towards the Deployment of Machine Learning Solutions in Network Traffic Classification: A Systematic Survey. IEEE Communications Surveys & Tutorials, v. 21, n. 2, p. 1988–2014, 2019.

RACHMAWATI, D. *et al.* Machine Learning Algorithm in Network Traffic Classification: A Performance Evaluation. Journal of Physics: Conference Series, v. 1899, n. 1, p. 012101, 2021.

SOMMER, R.; PAXSON, V. Outside the Closed World: On Using Machine Learning for Network Intrusion Detection. *In*: IEEE Symposium on Security and Privacy. Oakland: IEEE Computer Society, 2010. p. 305–316.

ZHANG, J. *et al.* Robust Network Traffic Classification Against Open-World Variants via Ensemble Learning. Computer Networks, v. 198, p. 108342, 2022.